

RAPPORT FINAL

Module : Sécurité NoSQL

Travaux Pratiques — Semestre S8

Année universitaire 2025–2026

Module NoSQL Security — S8

Table des matières

1. Introduction
2. Présentation des technologies utilisées
3. Description des TPs
 - 3.1 TP1 — Introduction MongoDB & noseclab
 - 3.2 TP2 — Modélisation SOC, Requêtes Avancées & Index
 - 3.3 TP3 — Authentification & Contrôle d'Accès (RBAC)
 - 3.4 TP4 — NoSQL Injection & Requêtes Sûres
4. Architecture des environnements
5. Mesures de sécurité appliquées
6. Vulnérabilités étudiées
7. Difficultés rencontrées
8. Conclusion

1. Introduction

Ce rapport présente l'ensemble des travaux pratiques réalisés dans le cadre du module **Sécurité NoSQL** du semestre S8. Ce module a pour objectif de former les étudiants à la manipulation, la modélisation et surtout la **sécurisation** de bases de données NoSQL, principalement MongoDB.

Les TP progressent du simple au complexe : de la découverte de l'environnement Docker/MongoDB à la mise en place de mécanismes d'authentification robustes (RBAC), puis à la sécurisation applicative contre les injections NoSQL. Chaque séance combine théorie et pratique dans un contexte réaliste de **Security Operations Center (SOC)**.

L'ensemble des travaux a été réalisé sous environnement Docker, garantissant la reproductibilité et l'isolation des expériences. Le projet est structuré de manière professionnelle avec documentation complète, captures d'écran et sauvegardes de données.

2. Présentation des technologies utilisées

2.1 MongoDB 7.0

MongoDB est un SGBD **NoSQL orienté document** qui stocke les données en BSON (Binary JSON). Il est largement utilisé dans la cybersécurité pour stocker des logs, alertes IDS, incidents et données de menaces.

Fonctionnalité	Description
Collections	Équivalent des tables SQL — regroupement de documents
Documents BSON	Format de stockage binaire basé sur JSON
Agrégations	Pipeline de traitement de données complexe
Index	Optimisation des requêtes fréquentes
RBAC	Contrôle d'accès basé sur les rôles
--auth	Activation de l'authentification obligatoire

2.2 Docker

Docker permet de créer des **conteneurs isolés** reproduisant exactement les mêmes environnements sur n'importe quelle machine. Chaque TP utilise un ou plusieurs conteneurs MongoDB, permettant de comparer des configurations sécurisées et vulnérables.

2.3 mongosh (MongoDB Shell)

mongosh est le shell interactif officiel de MongoDB. Il permet d'exécuter des commandes JavaScript sur la base : création de collections, insertion, agrégations, gestion des utilisateurs et rôles.

2.4 Node.js & Express (TP4)

Node.js est l'environnement d'exécution JavaScript côté serveur utilisé en TP4 pour construire l'API intermédiaire entre l'utilisateur et MongoDB. Express gère les routes HTTP (**POST /auth/login**, **POST /logs/search**). La sécurité applicative du TP4 repose entièrement sur la qualité du code Node.js/Express, démontrant que le hardening infrastructure (TP3) est insuffisant sans sécurisation de l'API.

3. Description des TPs

3.1 TP1 — Introduction MongoDB & noseclab

Le premier TP constitue une **prise en main complète** de l'environnement. L'objectif est de déployer MongoDB dans Docker et d'explorer une base de logs de sécurité : **noseclab**.

Objectifs atteints

- Déploiement d'un conteneur MongoDB 7.0 via Docker
- Connexion et navigation avec mongosh
- Exploration de la collection security_logs
- Premières requêtes de filtrage et de comptage
- Compréhension de la structure documentaire NoSQL vs SQL

Commandes clés

```
docker run -d --name mongo-noseclab -p 27017:27017 mongo:7
docker exec -it mongo-noseclab mongosh
use noseclab
db.security_logs.countDocuments()
db.security_logs.find({ level: 'ERROR' })
```

Résultats

La base noseclab contient une collection **security_logs** avec des documents structurés représentant des événements de sécurité : timestamp, IP source, niveau de sévérité, action et message. Ce TP a permis de comprendre les différences fondamentales entre les bases relationnelles et documentaires.

3.2 TP2 — Modélisation SOC, Requêtes Avancées & Index

Le deuxième TP porte sur la **conception et le peuplement d'une base SOC**. La base **soc_lite** simule un environnement réel avec plusieurs collections interconnectées.

Modèle de données

Collection	Contenu	Champs principaux
users	Analystes SOC	username, role, email, clearance_level
security_logs	Logs d'événements	timestamp, source_ip, event_type, severity
incidents	Incidents ouverts	incident_id, title, severity, status, assigned_to
ids_alerts	Alertes IDS/IPS	alert_type, source_ip, protocol, timestamp

Requêtes avancées réalisées

- Agrégation par sévérité des incidents (pipeline \$group)
- Recherche des incidents critiques non résolus
- Classement des alertes IDS par fréquence (\$sort, \$limit)
- Jointure simulée entre collections (\$lookup)
- Création d'index composés pour optimiser les requêtes SOC

Pipeline d'agrégation

```
db.incidents.aggregate([
  { $group: { _id: '$severity', count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

Index créés

```
db.security_logs.createIndex({ source_ip: 1 })
db.incidents.createIndex({ status: 1, severity: 1 })
db.security_logs.createIndex({ timestamp: -1 })
```

3.3 TP3 — Authentification & Contrôle d'Accès (RBAC)

Le troisième TP aborde la **sécurisation de MongoDB**. Il compare un déploiement insécurisé (sans authentification) à un déploiement sécurisé avec **--auth**, puis met en place un système RBAC complet adapté au contexte SOC.

Phase 1 — Démonstration de vulnérabilité

Un conteneur MongoDB **sans authentification** (mongo_insecure, port 27018) est lancé. Il est démontré qu'**aucun credential n'est requis** pour insérer, lire ou supprimer des données — vulnérabilité critique en production.

Phase 2 — Sécurisation avec --auth

```
docker run -d --name mongo_secure1 -p 27019:27017 mongo:7 --auth

use admin
db.createUser({
  user: 'admin', pwd: 'AdminSecure2026!',
  roles: [{ role: 'userAdminAnyDatabase', db: 'admin' }]
})
```

Utilisateurs et rôles SOC

Utilisateur	Rôle MongoDB	Droits
soc_app	readWrite	Lecture + écriture sur soc_lite
soc_analyst_ro	read	Lecture seule sur soc_lite
soc_manager	incidentManager (custom)	CRUD uniquement sur incidents

Tests de violation de droits

- soc_analyst_ro — tentative d'insertion → **MongoServerError[Unauthorized]**
- soc_manager — tentative de lecture des logs → **MongoServerError[Unauthorized]**
- soc_app — insertion de log → **Succès (readWrite autorisé)**

3.4 TP4 — NoSQL Injection & Requêtes Sûres

Le quatrième TP traite de la **sécurité applicative** : comment une API Node.js/Express mal sécurisée peut devenir un vecteur d'attaque contre MongoDB, même lorsque le hardening infrastructure (TP3) est correctement appliqué.

Le hardening infrastructure (authentification SCRAM, RBAC, restriction réseau) protège MongoDB contre une connexion directe non autorisée. Mais dans la réalité, une API fait l'intermédiaire — si elle passe les données utilisateur directement dans un **find()** sans validation, l'attaquant manipule l'API légitime sans jamais toucher MongoDB directement.

Objectifs du TP4

Objectif	Description
COMPRENDRE	Expliquer ce qu'est une injection NoSQL et en quoi elle diffère d'une injection SQL
IDENTIFIER	Repérer dans un code Node.js les points où une entrée utilisateur peut contaminer un filtre MongoDB
REPRODUIRE	Réaliser deux types d'injections NoSQL (contournement d'auth, exfiltration)
APPLIQUER	Mettre en place le pattern safeFilter pour construire une requête MongoDB sécurisée
CONFIGURER	Définir une projection minimale afin que l'API ne retourne que les champs nécessaires

3.4.1 Comprendre l'injection NoSQL

Une injection NoSQL est une attaque où un attaquant insère des opérateurs ou des structures JSON dans une entrée applicative afin de modifier la logique d'une requête MongoDB. Contrairement à l'injection SQL (qui casse une chaîne), l'injection NoSQL exploite le fait que les filtres MongoDB sont déjà des **objets JSON** — pas de cassure de syntaxe nécessaire.

Aspect	SQL Injection	NoSQL Injection (MongoDB)
Vecteur	Chaîne concaténée dans une requête SQL	Objet JSON inséré dans un filtre find()
Payload typique	' OR '1'='1	{'\$ne':''} ou {'\$regex':'.*'}
Mécanisme	Cassure de syntaxe SQL	Injection d'opérateurs dans l'objet JSON
Défense principale	Requêtes préparées / paramétrées	Validation de type + construction explicite (safeFilter)

Erreur fréquente : Beaucoup de développeurs pensent que MongoDB les protège naturellement des injections car "ce n'est pas du SQL". C'est une idée reçue dangereuse. La vulnérabilité n'est pas dans la base de données — elle est dans le code applicatif qui construit les filtres sans les valider. MongoDB exécute fidèlement le filtre qu'il reçoit.

Endpoint vulnérable typique

```
// POST /logs/search – VERSION VULNÉRABLE
app.post('/logs/search', async (req, res) => {
  const results = await db.collection('security_logs')
    .find(req.body) // req.body passé DIRECTEMENT – DANGEREUX
    .limit(20).toArray();
  res.json({ count: results.length, data: results });
});
```

- **Cas normal** : l'utilisateur envoie {"src_ip":"192.168.1.10"} → MongoDB filtre les logs de cette IP.
- **Cas injecté** : l'attaquant envoie {"src_ip":{"\$regex":""}} → regex vide = tous les logs retournés.

3.4.2 Démonstration des injections

Injection \$ne — Contournement d'authentification

L'opérateur {"champ":{"\$ne":valeur}} sélectionne tous les documents où champ est différent de valeur. Injecté à la place d'un mot de passe, il contourne l'authentification :

```
// Ce que l'API vulnérable construit involontairement :
{ username: "alice", password: { $ne: "" } }

// Ce que MongoDB comprend :
// "retourne alice dont le mot de passe est DIFFÉRENT de ''"
// = alice tout court (puisque'elle a un mot de passe)
// → Login réussi SANS connaître le mot de passe !
```

Impact SOC : L'attaquant n'a pas besoin du mot de passe d'Alice. L'API répond success:true, role:analyst. Toutes les actions suivantes sont journalisées sous l'identité d'Alice — la traçabilité des incidents est corrompue.

Injection \$regex et \$gt — Exfiltration de données

Opérateur	Description	Payload injecté	Impact
\$regex	Correspond à l'expression régulière. Regarde si le champ correspond à l'expression.	{src_ip:{"\$regex":""}}	Retourne tous les logs
\$gt	Supérieur à la valeur. Pour les chaînes, \$gt retourne la chaîne non vide.	{action:{"\$gt":""}}	Retourne tous les logs

Mass Assignment — Injection de champs non autorisés

Se produit quand une application applique directement un objet utilisateur à un \$set sans filtrer les champs modifiables :

```
// VULNÉRABLE
await db.collection('users').updateOne(
  { username: req.user.username },
  { $set: req.body } // TOUT req.body appliqué sans filtre !
);

// Attaquant envoie : {"email":"hacker@evil.com", "role":"admin"}
// → role passe à 'admin' (élévation de privilège silencieuse !)
```

3.4.3 Correction : Validation, Allow-list et Filtre Sûr

Trois défenses complémentaires à appliquer systématiquement avant toute requête MongoDB construite depuis une entrée utilisateur :

1 — Validation des types

```
function isCleanString(val) {
  if (typeof val !== 'string') return false;
  if (val.trim().length === 0) return false;
  if (val.length > 100) return false;
}
```



```

    return true;
}

function isValidIPv4(val) {
  if (typeof val !== 'string') return false;
  return /^(\\d{1,3}\\.){3}\\d{1,3}$/.test(val) && val.split('.').every(n => +n <= 255);
}

```

2 — Filtrage des opérateurs MongoDB (\$-filtering)

```

function hasNoMongoOperators(obj) {
  if (typeof obj !== 'object' || obj === null) return true;
  for (const key of Object.keys(obj)) {
    if (key.startsWith('$')) return false;
    if (!hasNoMongoOperators(obj[key])) return false;
  }
  return true;
}

```

3 — Le pattern safeFilter

```

// POST /logs/search – VERSION CORRIGÉE
const ALLOWED_STATUSES = ['open', 'closed', 'investigating'];

app.post('/logs/search', async (req, res) => {
  const safeFilter = {};

  if (req.body.src_ip !== undefined) {
    if (!isValidIPv4(req.body.src_ip))
      return res.status(400).json({ error: 'src_ip invalide – format IPv4 requis' });
    safeFilter.src_ip = req.body.src_ip;
  }

  if (req.body.status !== undefined) {
    if (!ALLOWED_STATUSES.includes(req.body.status))
      return res.status(400).json({ error: 'status non autorisé' });
    safeFilter.status = req.body.status;
  }

  const results = await db.collection('security_logs')
    .find(safeFilter)
    .project({ _id:0, ts:1, src_ip:1, action:1, status:1 })
    .limit(20).toArray();
  res.json({ count: results.length, data: results });
});

```

Endpoint /auth/login sécurisé

```

app.post('/auth/login', async (req, res) => {
  if (typeof req.body.username !== 'string' || typeof req.body.password !== 'string')
    return res.status(400).json({ error: 'username et password doivent être des chaînes' });

  const username = req.body.username.trim();
  const password = req.body.password;
  if (!isCleanString(username) || password.length === 0)
    return res.status(400).json({ error: 'Entrée invalide' });
}

```

```
const user = await db.collection('users').findOne(
  { username },
  { projection: { _id:0, username:1, password:1, role:1 } }
);

// Comparaison JS — ne peut PAS être manipulée par $ne
if (!user || user.password !== password)
  return res.status(401).json({ success: false, message: 'Identifiants incorrects' });
res.json({ success: true, message: `Bienvenue ${user.username}`, role: user.role });
});
```

3.4.4 Projection minimale et principe « Least Data »

Même si les injections sont bloquées, une API sans projection expose des données sensibles inutilement. La projection minimale consiste à déclarer explicitement les champs à inclure :

```
// Sans projection → FUIE de données
db.collection('users').findOne({ username: 'alice' })
// Retourne password, internal_notes, _id...

// Avec projection minimale
db.collection('users').findOne(
  { username: 'alice' },
  { projection: { _id:0, username:1, role:1 } }
)
// Retourne seulement : { username: 'alice', role: 'analyst' }
```

3.4.5 Résultats des tests

Test	Requête	Résultat	Statut
Injection \$ne — Login vulnérable	{'username':'alice','password':{'\$ne':'password'}}	HTTP 200 — succès — contournement réussi	Vulnérable
Injection \$ne — Login corrigé	Même requête sur API sécurisée	HTTP 400 — type invalide	Bloqué
Injection \$regex — Logs vulnérable	{'typeip':{'\$regex':''}}	Exfiltration de tous les logs	Vulnérable
Injection \$regex — Logs corrigé	Même requête sur API sécurisée	HTTP 400 — format IPv4 requis	Bloqué
Requête légitime — Login	{'username':'alice','password':'motdepasse123'}	HTTP 200 — succès — dépasse 123	Fonctionnel
Projection minimale	Requête avec password_hash	Champs sensibles absents	Masqués

3.4.6 Architecture de l'environnement TP4

Conteneur	Image	Port	Rôle
mongo-secure1	mongo:7	27019	Base MongoDB avec --auth (RBAC), réutilisée du TP3
soc-api-vulnerable	node:18	3000	API Express vulnérable — démonstration des injections
soc-api-secure	node:18	3001	API Express corrigée — pattern safeFilter

3.4.7 Vulnérabilités étudiées

Vulnérabilité	Opérateur	Impact	Mitigation
---------------	-----------	--------	------------

Contournement de login	<code>{ '\$ne': '' }</code>	Accès sans mot de passe, traçabilité	Validation de type + comparaison JS
Exfiltration de collection	<code>{ '\$regex': '' }</code>	Fuite de tous les documents	Validation de format + <code>safeFilter</code>
Sélection étendue	<code>{ '\$gt': '' }</code>	Résultats non filtrés	Validation de type + allow-list
Mass Assignment	Champs injectés dans <code>\$set</code>	Élévation de privilège silencieuse	Allow-list des champs modifiables
Fuite de champs	Absence de projection	Exposition de <code>password_hash</code> , <code>integrity</code>	Projection minimale sur tous les <code>find()</code>

3.4.8 Checklist Anti-NoSQLi

#	Règle	Implémentation
1	Valider le type de chaque entrée	<code>typeof val === 'string'</code> — rejeter les objets
2	Ne jamais faire <code>find(req.body)</code>	Construire un <code>safeFilter</code> champ par champ
3	Allow-list des valeurs	<code>Array.includes()</code> ou enum pour les champs à domaine fini
4	Valider les formats par regex	IPv4, dates ISO, UUIDs, emails
5	Filtrer les clés <code>\$</code>	Toute clé commençant par <code>\$</code> = tentative d'injection
6	Projection minimale	Exclure <code>_id:0</code> , <code>password_hash</code> , tout champ non nécessaire
7	Ne jamais exposer <code>err.message</code>	Message générique client, détail complet en log serveur
8	Mises à jour contrôlées	<code>\$set</code> avec champs nommés explicitement — jamais <code>\$set:req.body</code>
9	En production	Utiliser <code>bcrypt</code> ou <code>Argon2</code> pour stocker et comparer les mots de passe

3.4.9 Synthèse — API Vulnérable vs API Sécurisée

Aspect	API Vulnérable (server.js)	API Sécurisée (server-safe.js)
Construction du filtre	<code>find(req.body)</code> — objet utilisateur direct	<code>find(safeFilter)</code> — objet construit par l'application
Validation	Aucune	Type → Format → Allow-list → <code>safeFilter</code>
Login	<code>findOne({username, password})</code> dans le filtre	<code>findOne({username})</code> + comparaison JS
Mise à jour	<code>\$set: req.body</code>	<code>\$set: {email: validatedEmail}</code> — champs nommés
Projection	Aucune — tous les champs exposés	<code>{_id:0, ts:1, src_ip:1, action:1, status:1}</code>
Réponse erreur	<code>res.json({error: err.message})</code> — fuite	<code>res.status(400).json({error: 'Entrée invalide'})</code>
Résultat face à <code>\$ne</code>	Contournement réussi	Rejet HTTP 400
Résultat face à <code>\$regex</code>	Exfiltration complète	Rejet HTTP 400
Mass Assignment	Élévation de privilège possible	Champs non prévus ignorés

3.4.10 Conclusion du TP4

Le TP4 a démontré que la **sécurité applicative** constitue une couche indispensable et non substituable au hardening infrastructure du TP3. Même avec une authentification SCRAM robuste et des rôles RBAC granulaires, une API qui passe directement `req.body` dans un `find()` reste vulnérable aux injections NoSQL.

Le pattern **safeFilter** — construire un filtre contrôlé champ par champ avec validation de type, format et allow-list — constitue la défense fondamentale. La **projection minimale** ajoute une couche de défense en profondeur indépendante : même en cas de contournement partiel, les champs sensibles (password_hash, internal_notes) restent invisibles dans la réponse API.

4. Architecture des environnements

4.1 Conteneurs Docker

Conteneur	Image	Port	Mode	TP(s)
mongo-noseclab	mongo:7	27017	Sans auth	TP1, TP2
mongo_insecure	mongo:7	27018	Sans auth (vulnérable)	TP3
mongo_secure1	mongo:7	27019	Avec --auth (RBAC)	TP3, TP4
soc-api-vulnerable	node:18	3000	API Express vulnérable	TP4
soc-api-secure	node:18	3001	API Express sécurisée	TP4

4.2 Bases de données

Base de données	Collections	TP
noseclab	security_logs	TP1
soc_lite	users, security_logs, incidents, ids_alerts	TP2, TP3, TP4

4.3 Docker Compose

Un fichier **docker-compose.yml** à la racine du projet permet de lancer l'ensemble des environnements en une seule commande :

```
docker-compose up -d
```

Les volumes Docker garantissent la persistance des données. Les sauvegardes BSON dans chaque dossier TP permettent de restaurer rapidement les jeux de données de démonstration.

5. Mesures de sécurité appliquées

5.1 Authentification

L'activation du mode **--auth** de MongoDB est la première barrière. Elle rend obligatoire la présentation de credentials valides pour toute connexion. Sans cette option, n'importe quel client réseau peut lire et écrire librement dans la base.

5.2 Contrôle d'accès basé sur les rôles (RBAC)

Le RBAC MongoDB permet d'appliquer le **principe du moindre privilège** :

- **read** : accès en lecture seule — pour les analystes SOC
- **readWrite** : lecture et écriture — pour les applications
- **userAdmin** : gestion des utilisateurs — pour les administrateurs
- **Rôles personnalisés** : droits granulaires sur une collection spécifique

5.3 Mots de passe forts

Les mots de passe respectent les bonnes pratiques : longueur ≥ 12 caractères, mélange majuscules/minuscules/chiffres/caractères spéciaux (ex : AppP@ss2026!, AdminSecure2026!).

5.4 Isolation réseau

Chaque conteneur est isolé dans son propre réseau Docker. En production, les ports MongoDB ne doivent jamais être exposés directement sur Internet (bindIp, firewall).

5.5 Sécurité applicative — safeFilter (TP4)

Mesure applicative indépendante de la configuration MongoDB :

- Validation de type : typeof systématique sur chaque entrée utilisateur
- Construction explicite du filtre : pattern safeFilter, champ par champ
- Allow-list des valeurs : enum pour les champs à domaine fini
- Filtrage des opérateurs \$: détection récursive des clés commençant par \$
- Projection minimale : {_id:0, ...} sur toutes les requêtes de lecture
- Messages d'erreur génériques : ne jamais exposer err.message au client

6. Vulnérabilités étudiées

6.1 MongoDB sans authentification

Risque	Impact	Mitigation
Accès sans auth	Exfiltration complète des données	Activer --auth
Port exposé publiquement	Accès depuis Internet	Firewall, bindIp
Privilèges excessifs	Élévation de privilèges	RBAC + moindre privilège
Logs non chiffrés	Interception des données	TLS/SSL (TP futur)

6.2 Injection NoSQL (TP4)

Vecteur	Opérateur	Impact	Mitigation
---------	-----------	--------	------------

Contournement de login	<code>{"\$ne":""}</code>	Accès sans mot de passe, traçabilité	Validation de type + comparaison JS
Exfiltration de collection	<code>{"\$regex":""}</code>	Fuite de tous les documents	Validation de format + <code>safeFilter</code>
Sélection étendue	<code>{"\$gt":""}</code>	Résultats non filtrés	Validation de type + allow-list
Mass Assignment	<code>\$set:req.body</code>	Élévation de privilège silencieuse	Allow-list des champs modifiables
Fuite de champs	Absence de projection	Exposition de données internes	Projection minimale systématique

7. Difficultés rencontrées

Difficulté	TP	Solution
Conflits de ports Docker	TP3	Port 27018 pour conteneur vulnérable, 27019 pour sécurisé
Encodage URL des mots de passe	TP3	Mots de passe avec @ encodés en %40 dans la chaîne de connexion
Syntaxe JavaScript mongosh	TP1–TP3	Adaptation par rapport aux commandes SQL
Compréhension du BSON	TP1–TP2	Format non lisible directement — utilisation de <code>mongorestore</code>
Rôles personnalisés MongoDB	TP3	Comprendre la structure <code>privilege/resource/actions</code>
Différence SQL vs NoSQLi	TP4	Adoption du pattern <code>safeFilter</code> comme équivalent conceptuel
Validation récursive des objets	TP4	Implémentation récursive avec <code>Object.keys()</code> et <code>typeof</code>
Comparaison de mots de passe	TP4	Recherche par username uniquement + comparaison JS côté serveur

8. Conclusion

Ce module NoSQL Security a permis d'acquérir une compréhension pratique et concrète des enjeux de sécurité liés aux bases de données NoSQL, en particulier MongoDB.

Les quatre premiers TP ont progressivement construit les compétences nécessaires :

- **TP1** : Manipulation basique de MongoDB, exploration de données de sécurité
- **TP2** : Modélisation d'une base SOC, requêtes d'agrégation, index de performance
- **TP3** : Hardening infrastructure — authentification SCRAM, RBAC granulaire, isolation réseau
- **TP4** : Sécurité applicative — injection NoSQL, pattern `safeFilter`, projection minimale, défense en profondeur

L'approche par comparaison (insécurisé vs sécurisé) s'est révélée particulièrement efficace pour comprendre l'importance de chaque mesure, que ce soit au niveau infrastructure (TP3) ou applicatif (TP4). Les bonnes pratiques identifiées sont directement applicables dans des contextes professionnels réels, notamment dans les équipes SOC utilisant MongoDB.

Les TPs à venir (TP5 à TP7) approfondiront d'autres aspects : **chiffrement TLS/SSL**, audit des accès, détection d'intrusions et intégration avec des outils SIEM.